Lecture 21 — DAGs, Topological Sort, DP on DAGs, Strongly Connected Components

> Year Curriculum · Prerequisites: Week 14–15 (DP), Week 16 (BFS/DFS), Week 17 (shortest paths) · Estimated time: 3 hours

1. Overview

Directed graphs model dependencies: course prerequisites, build steps, one-way roads, transitions between game states. Two structural questions dominate:

1. **Is there a cycle?** If not, the graph is a *directed acyclic graph* (DAG) and its vertices can be listed in a **topological order** in which every edge points forward. Along that order we can run dynamic programming — longest paths, path counts, minimum costs — in linear time, even with negative weights.
2. **If there are cycles, what is their structure?** Grouping vertices into **strongly connected components** (SCCs) collapses every cycle into a single node. The resulting *condensation* graph is always a DAG, so the techniques of part 1 apply again.

This lecture also makes precise a remark from Week 15: **every dynamic program is a computation on a DAG of states**.

2. DAGs and topological order

A **topological order** of a directed graph is an ordering of its vertices such that for every edge u → v, u comes before v.

Theorem. A directed graph has a topological order if and only if it has no directed cycle.

- If there is a cycle $v_1 \to v_2 \to \dots \to v_k \to v_1$, each vertex must come before the next, so v_1 would come before itself — impossible.
- If there is no cycle, some vertex has in-degree 0 (otherwise walk backwards along incoming edges forever; with finitely many vertices you would revisit one, forming a cycle). Place it first, delete it, and repeat on the remaining graph, which is still acyclic.

That second argument *is* Kahn's algorithm. A DAG typically has many topological orders; most problems accept any of them.

2.1 Kahn's algorithm

Compute in-degrees. Put all in-degree-0 vertices in a queue. Repeatedly pop a vertex, append it to the order, and decrement the in-degree of each out-neighbour; when a neighbour's in-degree reaches 0, push it.

If the final order contains fewer than n vertices, the leftover vertices all have positive in-degree among themselves, so they contain a cycle — print IMPOSSIBLE. Time O(n + m).

Trace — CSES 1679 *Course Schedule*. Edges 1→2, 3→1, 4→5. Initially vertices 3 and 4 have in-degree 0. Pop 3 (vertex 1 drops to 0, push it), pop 4 (push 5), pop 1 (push 2), pop 5, pop 2. Order: `3 4 1 5 2`.

Variants. Replace the queue with a min-heap (`priority_queue` with `greater<>`) to obtain the lexicographically smallest topological order. Kahn's algorithm processed in "rounds" (all current in-degree-0 vertices form a round) gives the minimum number of semesters needed when any number of courses can be taken in parallel.

2.2 DFS-based topological sort

Run DFS from every unvisited vertex and append each vertex to a list when it **finishes** (after all its descendants are done). The reversed list is a topological order.

Why: consider an edge u → v in a DAG. If DFS reaches v while exploring u, v finishes first. If v was already finished, v finished first too. And v cannot be "in progress" (on the recursion stack) when we look at u → v, because then there would be a path v ⇝ u, and with u → v a cycle. So v always finishes before u, and in reversed finishing order u precedes v.

2.3 Detecting a directed cycle

Colour vertices **white** (unvisited), **gray** (on the current recursion stack) and **black** (finished). During DFS, an edge to a **gray** vertex is a *back edge* and closes a directed cycle. An edge to a black vertex is harmless (it leads to an already-finished part of the graph).

A single boolean "visited" array is not enough for directed graphs: in 1 → 2, 1 → 3, 3 → 2, vertex 2 is visited when we reach it from 3, yet there is no cycle. (For *undirected* graphs, Week 16's "visited and not my parent" rule is correct; do not mix the two.)

To *print* a cycle (CSES 1678 *Round Trip II*), keep DFS parents; when you find a back edge u → v, walk the parents from u back to v.

3. Dynamic programming on DAGs

In a topological order, every predecessor of v is processed before v. So any recurrence of the form "value of v depends on values of its predecessors" can be evaluated in a single pass. Equivalently, process in *reverse* order for recurrences over successors.

3.1 Longest path — CSES 1680 *Longest Flight Route*

Find a route from city 1 to city n visiting the maximum number of cities, in a graph that is guaranteed acyclic. `best[v]` = maximum number of cities on a route from 1 to v; `best[1] = 1` and $-\infty$ for everything else initially. In topological order, for each u with a finite `best[u]`, relax each edge u → v by `best[u] + 1`, storing a parent pointer for reconstruction.

The $-\infty$ initialisation is important: a vertex that is not reachable from 1 must not be used as the start of a route. Initialising with 0 instead would silently allow routes that begin elsewhere.

Longest simple path is NP-hard in general graphs, but in a DAG every path is simple, so this O(n + m) DP is exact. Shortest paths in a DAG work the same way with `min`, and **negative weights pose no problem** — no Bellman-Ford needed.

3.2 Counting paths — CSES 1681 *Game Routes*

Number of routes from 1 to n modulo 10^9+7: `ways[1] = 1`, and in topological order `ways[v] += ways[u]` for each edge u → v. Answer `ways[n]`.

3.3 Every DP is a DAG

Take any DP: its states are vertices; "state X is computed from state Y" is an edge Y → X; base cases are sources. The DP is well defined precisely because this graph has no cycle, and any valid evaluation order is a topological order:

- Grid paths (Week 15): cells with edges right/down; row-major order is topological.
- Knapsack: states (i, c) with edges to (i+1, c) and (i+1, c+w_i).
- Memoised recursion performs a DFS on the state DAG and computes each state when it *finishes* — exactly the DFS topological sort.

The converse lesson matters too: if your "DP" has cyclic dependencies (for example, a grid where moves go in all four directions), it is not a DP. It is a shortest-path problem and needs BFS or Dijkstra (Weeks 16–17).

4. Strongly connected components

Vertices u and v are **strongly connected** if each can reach the other. This is an equivalence relation, and its classes are the **strongly connected components**. Every directed cycle lies entirely within one SCC. Contracting each SCC to a single node, and keeping an edge between two SCCs whenever some original edge joins them, gives the **condensation**, which is always a DAG (a cycle among SCCs would merge them into one).

Example: edges 1→2, 2→3, 3→1, 3→4, 4→5, 5→4 have SCCs {1, 2, 3} and {4, 5}, and the condensation is a single edge {1,2,3} → {4,5}.

4.1 Kosaraju's algorithm

1. Run DFS over the whole graph G and record vertices in order of **finishing time**.
2. Build the reversed graph G^R (every edge flipped).
3. Go through vertices in **decreasing** finishing time. Each time you meet a vertex not yet assigned, run a DFS (or BFS) in G^R from it; every newly reached vertex forms one SCC.

Why it works. The vertex with the latest finishing time lies in a SCC C that is a **source** of the condensation (nothing outside C reaches C). In G^R, C becomes a sink: a search from inside C cannot leave C, and it reaches all of C because C is strongly connected. Remove C and repeat; the next-latest finisher lies in a source of the remaining condensation. (The general statement: if there is an edge from SCC C to SCC D, then the maximum finishing time in C exceeds that in D.)

Kosaraju uses two linear passes, O(n + m), and is easy to implement iteratively (reference code §5.3), which makes it robust against recursion-depth limits. The components are discovered in topological order of the condensation.

4.2 Tarjan's algorithm

Tarjan finds SCCs in a single DFS. Each vertex gets a discovery time `tin[u]` and a value `low[u]`: the smallest discovery time reachable from u's DFS subtree using tree edges plus at most one edge to a vertex still on the algorithm's stack. Vertices are pushed onto an explicit stack when discovered.

- For a tree edge u → v: recurse, then `low[u] = min(low[u], low[v])`.
- For an edge u → v where v is on the stack: `low[u] = min(low[u], tin[v])`.
- Edges to vertices that are visited but **not** on the stack lead into SCCs that are already complete; ignore them.
- After processing u, if `low[u] == tin[u]`, u is the first-discovered vertex (the root) of its SCC: pop the stack down to u; those vertices form the SCC.

Tarjan emits SCCs in **reverse topological order** of the condensation: when an SCC is popped, every SCC it can reach has already been popped. Numbering SCCs 0, 1, 2, … in emission order therefore guarantees every condensation edge goes from a larger id to a smaller one, so a DP over successors can simply loop ids in increasing order.

Tarjan is recursive in its natural form; with $n = 10^5$ the recursion depth can reach 10^5, fine on CSES and Codeforces. The same `tin`/`low` machinery finds bridges and articulation points, covered in the National track.

4.3 Condensation + DAG DP — CSES 1686 *Coin Collector*

Rooms contain coins; tunnels are one-way; you may start and end anywhere and revisit rooms. Within an SCC you can visit every room and return, so you collect the whole SCC's coins. Between SCCs, movement follows the condensation DAG. The answer is the maximum vertex-weighted path in the condensation:

```
best[C] = val[C] + max(0, max over edges C → D of best[D])
answer  = max over C of best[C]
```

With Tarjan's numbering, successors have smaller ids, so loop C = 0, 1, …. On the sample (coins 4 5 2 7; edges 1→2, 2→1, 1→3, 2→4) the SCC {1, 2} is worth 9 and leads to {3} (2) or {4} (7), so the answer is 16.

4.4 More SCC applications

- **CSES 1682 *Flight Routes Check*.** Is every city reachable from every other? Equivalent to "1 reaches all vertices in G, and 1 reaches all vertices in G^R" — two BFS passes, no SCC algorithm needed. If not, print a vertex not reached and the direction that fails.
- **Making a digraph strongly connected.** If the condensation has more than one node, with s sources and t sinks, the minimum number of edges to add is max(s, t).
- **2-SAT** (National track): build the implication graph; the formula is satisfiable iff no variable x is in the same SCC as ¬x.

5. Reference implementations

All compile with `g++ -std=c++17 -O2 -Wall` and pass the CSES samples; the SCC and Coin Collector programs were also compared against a brute-force reachability check on random graphs.

5.1 Kahn's algorithm — CSES 1679 *Course Schedule*

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;                      // course a before course b
        adj[a].push_back(b);
        indeg[b]++;
    }
    queue<int> q;
    for (int v = 1; v <= n; v++)
        if (indeg[v] == 0) q.push(v);       // no prerequisites left
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u])
            if (--indeg[v] == 0) q.push(v);
    }
    if ((int)order.size() < n) { cout << "IMPOSSIBLE\n"; return 0; }   // a cycle remains
    for (int v : order) cout << v << " ";
    cout << "\n";
}
```

5.2 Longest path in a DAG — CSES 1680 *Longest Flight Route*

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> adj(n + 1);
    vector<int> indeg(n + 1, 0);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        adj[a].push_back(b);
        indeg[b]++;
    }
    queue<int> q;
    for (int v = 1; v <= n; v++) if (indeg[v] == 0) q.push(v);
    vector<int> order;
```

```
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
    }
    const int NEG = INT_MIN;
    vector<int> best(n + 1, NEG), par(n + 1, 0);   // best[v] = most cities on a 1 -> v
route
    best[1] = 1;
    for (int u : order) {                  // topological order: best[u] is final here
        if (best[u] == NEG) continue;      // unreachable from 1
        for (int v : adj[u])
            if (best[u] + 1 > best[v]) { best[v] = best[u] + 1; par[v] = u; }
    }
    if (best[n] == NEG) { cout << "IMPOSSIBLE\n"; return 0; }
    vector<int> path;
    for (int v = n; v != 1; v = par[v]) path.push_back(v);
    path.push_back(1);
    reverse(path.begin(), path.end());
    cout << path.size() << "\n";
    for (int v : path) cout << v << " ";
    cout << "\n";
}
```

For *Game Routes* (§3.2), keep the same skeleton and replace the relaxation by `ways[v] = (ways[v] + ways[u]) % MOD`.

5.3 Kosaraju, iterative — CSES 1683 *Planets and Kingdoms*

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<vector<int>> g(n + 1), rg(n + 1);
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
        rg[b].push_back(a);                // reversed graph
    }
    // Pass 1: DFS on g, record vertices by finishing time
    vector<int> finish;
    vector<bool> vis(n + 1, false);
    vector<size_t> it(n + 1, 0);           // next neighbour index per vertex
    for (int s = 1; s <= n; s++) {
        if (vis[s]) continue;
        vector<int> st = {s};
        vis[s] = true;
        while (!st.empty()) {
```

```
            int u = st.back();
            if (it[u] < g[u].size()) {
                int v = g[u][it[u]++];
                if (!vis[v]) { vis[v] = true; st.push_back(v); }
            } else {                        // all neighbours done: u finishes
                finish.push_back(u);
                st.pop_back();
            }
        }
    }
    // Pass 2: on the reversed graph, in decreasing finishing time
    vector<int> comp(n + 1, 0);
    int k = 0;
    for (int i = n - 1; i >= 0; i--) {
        int s = finish[i];
        if (comp[s]) continue;
        k++;
        vector<int> st = {s};
        comp[s] = k;
        while (!st.empty()) {
            int u = st.back(); st.pop_back();
            for (int v : rg[u])
                if (!comp[v]) { comp[v] = k; st.push_back(v); }
        }
    }
    cout << k << "\n";
    for (int v = 1; v <= n; v++) cout << comp[v] << " ";
    cout << "\n";
}
```

Note the iterative pass 1: it keeps a per-vertex iterator `it[u]` so that a vertex is popped (finished) only after *all* its neighbours have been explored. The simpler "pop and push all neighbours" stack from Week 16 does **not** produce correct finishing times.

5.4 Tarjan + condensation DP — CSES 1686 *Coin Collector*

```
#include <bits/stdc++.h>
using namespace std;
int n, m, timer_ = 0, nComp = 0;
vector<vector<int>> g;
vector<int> tin, low, comp, stk;
vector<bool> onStack;
void tarjan(int u) {
    tin[u] = low[u] = ++timer_;
    stk.push_back(u);
    onStack[u] = true;
    for (int v : g[u]) {
        if (!tin[v]) {                      // tree edge
            tarjan(v);
            low[u] = min(low[u], low[v]);
        } else if (onStack[v]) {            // edge back into the current SCC
            low[u] = min(low[u], tin[v]);
```

```
        }
    }
    if (low[u] == tin[u]) {                // u is the root of an SCC: pop it
        while (true) {
            int v = stk.back(); stk.pop_back();
            onStack[v] = false;
            comp[v] = nComp;
            if (v == u) break;
        }
        nComp++;
    }
}
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    vector<long long> coins(n + 1);
    for (int i = 1; i <= n; i++) cin >> coins[i];
    g.assign(n + 1, {});
    for (int i = 0; i < m; i++) {
        int a, b;
        cin >> a >> b;
        g[a].push_back(b);
    }
    tin.assign(n + 1, 0); low.assign(n + 1, 0); comp.assign(n + 1, -1);
    onStack.assign(n + 1, false);
    for (int v = 1; v <= n; v++) if (!tin[v]) tarjan(v);
    // Tarjan numbers SCCs in REVERSE topological order: edges go from higher id to lower
id
    vector<long long> val(nComp, 0), best(nComp, 0);
    for (int v = 1; v <= n; v++) val[comp[v]] += coins[v];
    vector<vector<int>> dag(nComp);
    for (int u = 1; u <= n; u++)
        for (int v : g[u])
            if (comp[u] != comp[v]) dag[comp[u]].push_back(comp[v]);
    long long ans = 0;
    for (int c = 0; c < nComp; c++) {      // successors (smaller ids) are already final
        best[c] = val[c];
        for (int d : dag[c]) best[c] = max(best[c], val[c] + best[d]);
        ans = max(ans, best[c]);
    }
    cout << ans << "\n";
}
```

6. Common mistakes

- **Not checking for leftovers in Kahn.** If `order.size() < n` there is a cycle; printing the partial order is wrong.
- **Directed cycle detection with a visited flag only.** You need three states (white/gray/black).

- **Wrong iterative DFS for finishing times.** The push-all-neighbours stack does not finish vertices in the right order; use per-vertex iterators.
- **Starting the DAG DP everywhere** when the problem fixes the source; initialise unreachable states to $-\infty$ (or a "not reachable" flag), not 0.
- **Tarjan without `onStack`.** Using `tin[v]` for edges into already-completed SCCs merges components that are not strongly connected.
- **Duplicate condensation edges.** Many original edges can map to the same pair of SCCs. Harmless for max/min DPs, but they double-count in path-counting DPs; deduplicate (sort + unique) when counting.
- **Overflow.** Coin sums reach $10^5 \cdot 10^9$; path counts need the modulus at every addition.
- **Recursion depth** in recursive DFS/Tarjan on long chains on judges with small stacks.

7. Practice problems

- CSES 1679 — Course Schedule — https://cses.fi/problemset/task/1679
- CSES 1678 — Round Trip II (print a directed cycle) — https://cses.fi/problemset/task/1678
- CSES 1680 — Longest Flight Route — https://cses.fi/problemset/task/1680
- CSES 1681 — Game Routes — https://cses.fi/problemset/task/1681
- CSES 1757 — Course Schedule II (lexicographic twist; think in reverse) — https://cses.fi/problemset/task/1757
- AtCoder DP Contest G — Longest Path — https://atcoder.jp/contests/dp/tasks/dp_g
- CSES 1682 — Flight Routes Check — https://cses.fi/problemset/task/1682
- CSES 1683 — Planets and Kingdoms — https://cses.fi/problemset/task/1683
- CSES 1686 — Coin Collector — https://cses.fi/problemset/task/1686
- Codeforces 510C — Fox And Names (build the graph, then toposort) — https://codeforces.com/problemset/problem/510/C
- Codeforces 427C — Checkposts (SCCs, min cost and count) — https://codeforces.com/problemset/problem/427/C

8. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapters 16 (Directed graphs: topological sorting, dynamic programming) and 17 (Strong connectivity, Kosaraju, 2-SAT).
- CP-Algorithms: "Topological Sorting" (https://cp-algorithms.com/graph/topological-sort.html) and "Strongly connected components and condensation graph" (https://cp-algorithms.com/graph/strongly-connected-components.html).
- R. E. Tarjan, "Depth-first search and linear graph algorithms", *SIAM Journal on Computing*, 1972.
- Looking ahead: Week 22 (LCA and Euler tours) uses DFS entry/exit times; the National track's advanced-graphs lecture builds bridges, articulation points and 2-SAT on the `tin`/`low` ideas from Tarjan.